Pattern Recognition 96 (2019) 106973

Contents lists available at ScienceDirect

Pattern Recognition

journal homepage: www.elsevier.com/locate/patcog

Learning representations of multivariate time series with missing data

Filippo Maria Bianchi[a,*], Lorenzo Livi[c,d], Karl Øyvind Mikalsen[a], Michael Kampffmeyer[a], Robert Jenssen[a,b]

[a] Machine Learning Group, UiT the Arctic University of Norway, Norway
[b] Norwegian Computing Center, Oslo, Norway
[c] Departments of Computer Science and Mathematics, University of Manitoba, Winnipeg, MB R3T 2N2, Canada
[d] Department of Computer Science, College of Engineering, Mathematics and Physical Sciences, University of Exeter, Exeter EX4 4QF, UK

ARTICLE INFO

Article history:
Received 29 August 2018
Revised 30 May 2019
Accepted 15 July 2019
Available online 19 July 2019

Keywords:
Representation learning
Multivariate time series
Autoencoders
Recurrent neural networks
Kernel methods

ABSTRACT

Learning compressed representations of multivariate time series (MTS) facilitates data analysis in the presence of noise and redundant information, and for a large number of variates and time steps. However, classical dimensionality reduction approaches are designed for vectorial data and cannot deal explicitly with missing values. In this work, we propose a novel autoencoder architecture based on recurrent neural networks to generate compressed representations of MTS. The proposed model can process inputs characterized by variable lengths and it is specifically designed to handle missing data. Our autoencoder learns fixed-length vectorial representations, whose pairwise similarities are aligned to a kernel function that operates in input space and that handles missing values. This allows to learn *good* representations, even in the presence of a significant amount of missing data. To show the effectiveness of the proposed approach, we evaluate the quality of the learned representations in several classification tasks, including those involving medical data, and we compare to other methods for dimensionality reduction. Successively, we design two frameworks based on the proposed architecture: one for imputing missing data and another for one-class classification. Finally, we analyze under what circumstances an autoencoder with recurrent layers can learn better compressed representations of MTS than feed-forward architectures.

© 2019 Elsevier Ltd. All rights reserved.

1. Introduction

Real-valued multivariate time series (MTS) allow to characterize the evolution of complex systems and is the core component in many research fields and application domains [1,2]. MTS analysis should account for relationships across variables and time steps, and, at the same time, deal with unequal time lengths [3–5]. Thanks to their ability to capture long-term dependencies, recurrent neural networks (RNNs) achieve state of the art results in tasks involving time series with one or more variables. In the latter case, RNNs can also be coupled with convolutional neural networks to explicitly model relationships across the different variables [6].

In time series analysis it is important to handle missing values, which are commonly found in real-world data such as electronic health records (EHR) [7,8], and are usually filled with imputation techniques in a pre-processing phase [9]. However, unless data are missing completely at random [10], imputation destroys useful information contained in the missingness patterns. Furthermore, an imputation method may introduce a strong bias that influences the outcome of the analysis, especially for large fractions of missing values [11]. A data-driven approach has recently been proposed to learn when to switch between two particular types of imputation [12], but it relies on strong assumptions that are suitable only for specific applications.

A proper representation of the data is crucial in machine learning applications [13]. While traditional approaches rely on manual feature engineering that requires time and domain expertise, representation learning aims at automatically producing features describing the original data [14]. Dimensionality reduction has been a fundamental research topic in machine learning [15–17] and its application in representation learning for extracting relevant information and generating compressed representations facilitates analysis and processing of data [3,18,19]. This is particularly important in the case of MTS, which often contain noise and redundant information with a large number of variables and time steps [19]. However, classical dimensionality reduction approaches are not

* Corresponding author.
E-mail addresses: filippo.m.bianchi@uit.no (F.M. Bianchi), l.livi@exeter.ac.uk (L. Livi), karl.o.mikalsen@uit.no (K.Ø. Mikalsen), michael.c.kampffmeyer@uit.no (M. Kampffmeyer), robert.jenssen@uit.no (R. Jenssen).
URL: http://site.uit.no/ml/ (F.M. Bianchi)

https://doi.org/10.1016/j.patcog.2019.106973
0031-3203/© 2019 Elsevier Ltd. All rights reserved.

designed to process sequential data, especially in the presence of missing values and variable input size.

1.1. Contributions

In this paper, we propose a novel neural network architecture, called the *Temporal Kernelized Autoencoder* (TKAE), to learn compressed representations of real-valued MTS that contain missing data and that may be characterized by unequal lengths. We assume data to be missing at random (MAR), i.e., the probability of data to be missing depends on the observed variables and the missingness patterns might be useful to characterize the data. Under the MAR assumption, once one has conditioned on all the data at hand, any remaining missingness is completely random (i.e., it does not depend on some unobserved variable). The MAR assumption covers many practical cases and includes also those situations where data are missing completely at random, i.e., the probability of missing data is independent from any other variable and there are no missingness patterns.

Our contributions are summarized in the following.

Learning compressed representations of MTS with missing data. To handle missing data effectively, and, at the same time, avoid the undesired biases introduced by imputation, we propose a *kernel alignment* procedure [20] that matches the dot product matrix of the learned representations with a kernel matrix. Specifically, we exploit the recently-proposed Time series Cluster Kernel (TCK) [21], which computes similarities between MTS with missing values without using imputation. By doing so, we generate representations that preserve unbiased pairwise similarities between MTS even in the presence of large amounts of missing data.

The encoder and the decoder in the TKAE are implemented by stacking multiple RNNs, allowing to generate a *fixed-size* vectorial representation of input MTS with variable-lengths. To better capture time dependencies, we use a bidirectional RNN [22] in the encoder. The final states of the forward and backward RNNs are combined by a dense nonlinear layer that reduces the dimensionality of the representation.

The proposed architecture serves different purposes. First of all, it transports the data from a complex input domain to a low-dimensional vector space while preserving the original relationships between inputs described by the TCK. Once represented as ordinary vectors, the MTS can then be processed by standard classifiers or by unsupervised machine learning algorithms [23], and their indexing and retrieval are more efficient [24,25]. Furthermore, when the dimensionality of the data is reduced, the models can potentially be trained with fewer samples.

Frameworks for missing data imputation and anomaly detection. Contrarily to other nonlinear dimensionality reduction techniques, the TKAE provides a decoder that yields an explicit mapping back to the input space. We exploit this feature to implement frameworks for two different applications. Specifically, we use the TKAE and its decoder to (i) implement an imputation method that leverages the generalization capability of the decoder reconstruction, rather than relying on *a-priori* assumptions that may introduce stronger biases, and (ii) design an anomaly detector based on the reconstruction error of the inputs.

Analysis of how RNNs encode MTS. We provide a detailed analysis of the effect of using RNNs for encoding MTS, which is the mechanism adopted in TKAE to handle inputs with variable lengths. Despite the popularity of AEs based on RNNs [26] for applications focused on text [27], speech [28], and video data [29], significantly fewer research efforts have been devoted so far in applying these architectures to real-valued MTS and, in general, in the context of dynamical systems. To fill this gap, we investigate under which circumstances recurrent architectures generate better compressed representations of MTS than feed-forward networks, which use padding to obtain inputs of equal length. Results show that the effectiveness of the RNNs is related to specific intrinsic properties of the MTS.

1.2. Paper organization

The paper is organized as follows. In Section 2 we first provide the background for existing AE models and the TCK. Then, we introduce the proposed TKAE architecture. In Section 3, we evaluate the TKAE's capability to learn good representations both on controlled tests and real-world MTS classification datasets. Results confirm that our method significantly improves the quality of the representations as the percentage of missing data increases. Successively, in Section 3.2 we propose two different frameworks that exploit the properties of the TKAE *decoder* for (i) imputing missing data and (ii) building a one-class classifier. We achieve competitive results for the imputation task, while we outperform other state-of-the-art methods in one-class classification. In Section 4, we perform an in-depth analysis to investigate which MTS are better represented by an AE with RNNs. We report several results obtained in controlled environments, as well as on benchmark data, to support the findings in our analysis. Finally, Section 5 reports our conclusions.

2. Methods

2.1. The autoencoder

The AE is a neural network traditionally conceived as a non-linear dimensionality reduction algorithm [30], which has been further exploited to learn representations in deep architectures [31] and to pre-train neural network layers [32]. An AE simultaneously learns two functions; the first one, called the *encoder*, is a mapping from an input domain, $\mathbb{R}^{D_x}$, to a hidden representation (*code*) in $\mathbb{R}^{D_z}$. The second function, the *decoder*, maps from $\mathbb{R}^{D_z}$ back to $\mathbb{R}^{D_x}$. The encoding function $\phi : \mathbb{R}^{D_x} \to \mathbb{R}^{D_z}$ and the decoding function $\psi : \mathbb{R}^{D_z} \to \mathbb{R}^{D_x}$ are defined as

$$\mathbf{z} = \phi(\mathbf{x}; \boldsymbol{\theta}_E); \ \tilde{\mathbf{x}} = \psi(\mathbf{z}; \boldsymbol{\theta}_D), \tag{1}$$

where $\mathbf{x} \in \mathbb{R}^{D_x}$, $\mathbf{z} \in \mathbb{R}^{D_z}$, and $\tilde{\mathbf{x}} \in \mathbb{R}^{D_x}$ denote a sample from the input space, its hidden representation, and its reconstruction given by the decoder, respectively. The encoder $\phi(\cdot)$ is usually implemented by stacking dense layers of neurons equipped with nonlinear activation functions. The decoder $\psi(\cdot)$ is architecturally symmetric to the encoder and operates in reverse direction; when inputs are real-valued vectors, the decoder's squashing nonlinearities are often replaced by linear activations [33]. Finally, $\boldsymbol{\theta}_E$ and $\boldsymbol{\theta}_D$ are the trainable parameters of the encoder and decoder, respectively. The parameters are the connection weights and biases of each layer m, i.e., $\boldsymbol{\theta}_E = \{\mathbf{W}_E^{(m)}, \mathbf{b}_E^{(m)}\}$ and $\boldsymbol{\theta}_D = \{\mathbf{W}_D^{(m)}, \mathbf{b}_D^{(m)}\}$. AEs are trained to minimize the discrepancy between the input $\mathbf{x}$ and its reconstruction $\tilde{\mathbf{x}}$. In the case of real-valued inputs, this is usually achieved by minimizing a loss L_r implemented as the empirical Mean Squared Error (MSE). It has been shown that for real-valued data, when the MSE between original and reconstructed input is minimized, the learned representations are *good* in the sense that the amount of mutual information with respect to the input is maximized [33].

In this paper, we focus on AEs with a "bottleneck", which learn an under-complete representation of the input, i.e., $D_z < D_x$, retaining as much useful information as possible to allow an accurate reconstruction [30]. The learned lossy, compressed representation of the input, can be exploited, e.g., for clustering and visualization tasks [34], or to train a classifier [35]. The bottleneck already provides a strong regularization as it limits the variance of the model.

However, further regularization can be introduced by tying the encoder and decoder weights ($\mathbf{W}_D = \mathbf{W}_E^T$) or by adding a ℓ_2-norm penalty to the loss function

$$L = L_r + \lambda L_2 = \mathrm{MSE}(\mathbf{x}, \tilde{\mathbf{x}}) + \lambda \|\mathbf{W}\|_2^2, \tag{2}$$

where L_2 is the ℓ_2-norm of all model weights $\mathbf{W} = \{\mathbf{W}_D, \mathbf{W}_E\}$ and λ is the hyperparameter controlling the contribution of the regularization term.

Recurrent neural networks (RNNs) are models excelling in capturing temporal dependencies in sequences [36,37] and are at the core of *seq2seq* models [26]. The latter, learns fixed-size representations of sequences with unequal length and, at the same time, generates variable-length outputs.

Modern *seq2seq* architectures implement a powerful mechanism called *attention*, which provides an inductive bias that facilitates the modeling of long-term dependencies and grants a more accurate decoding if the lengths of the input sequences varies considerably [38,39]. However, models with attention provide a representation that is neither compact nor of fixed size and, henceforth, are not suitable for our purposes. If fact, rather than learning a single vector representation for the whole input sequence, a model with attention maintains all the encoder states generated over time, which are combined by a time-varying *decoding vector* at each decoding step.

2.2. The time series cluster kernel

The time series cluster kernel (TCK) [21] is an algorithmic procedure to compute unsupervised kernel similarities among MTS containing missing data. The TCK is able to model time series with missing data, under the MAR assumption. The method is grounded on an ensemble learning approach that guarantees robustness with respect to hyperparameters. This ensures that the TCK works well in unsupervised settings (the ones in which AEs actually operate), where it is not possible to tune hyperparameters by means of supervised cross-validation. The base models in the ensemble are Gaussian mixture models (GMMs), whose components are fit to the dataset. By fitting GMMs with different numbers of mixtures, the TCK procedure generates partitions at different resolutions that capture both local and global structures in the data.

To further enhance diversity in the ensemble, each partition is evaluated on a random subset of MTS samples, attributes (variates), and time segments, using random initializations and randomly chosen hyperparameters. This also contributes to provide robustness with respect to hyperparameters, such as the number of mixture components. To avoid imputation, missing data are analytically marginalized away in the likelihoods. To obtain the GMM posteriors, the likelihoods are multiplied with smooth priors, whose contribution becomes stronger as the percentage of missingness increases. The TCK is then built by summing up, for each partition, the inner products between pairs of posterior assignments corresponding to different MTS. More details on the TCK are provided in Appendix A.

2.3. The temporal kernelized autoencoder

The *Temporal Kernelized Autoencoder* (TKAE) is our proposed AE architecture, which is specifically conceived to learn compressed representations of variable-length MTS that may contain missing values. A schematic depiction of the TKAE is provided in Fig. 1.

We assume that each MTS can be represented by a matrix $\mathbf{X} \in \mathbb{R}^{V \times T}$, where V denotes the number of variates and T is the number of time steps that may vary in each MTS. Analogously to *seq2seq* [26], in TKAE the dense layers of standard AEs are replaced by RNNs, which process inputs sequentially and update their hidden state at each time step t according to the following mapping,

$$\mathbf{h}_t = \phi(\mathbf{x}_t, \mathbf{h}_{t-1}, \boldsymbol{\theta}_E), \tag{3}$$

where $\boldsymbol{\theta}_E$ is the set of trainable parameters of the recurrent units. The recurrent layers are composed of either gated recurrent units (GRU) [40] or long short-term memory (LSTM) [41] cells. The choice of the cell is usually guided by the task at hand [42].

Conventional RNNs make use of previous inputs to build their current hidden representation [22]. However, in applications like MTS classification where the whole input is available at once, it is also possible to exploit the information contained in future inputs to generate the current state of the network. For this reason, the encoder is implemented as a stacked bidirectional RNN [43] consisting of two RNNs working in parallel, each one with M layers of D_z cells and transition function (3). One of the RNNs captures input dependencies going *backward* in time, whereas the other processes the same input but in reverse order, thus modeling relationships going *forward* in time. After the whole input is processed, the final states of the forward and backward RNN are denoted as $\mathbf{h}_T^f$ and $\mathbf{h}_T^b$, respectively. While $\mathbf{h}_T^f$ is influenced by past observations, $\mathbf{h}_T^b$ depends on future ones. Hence, their combination can capture a wider range of temporal dependencies in the input. In the TKAE, dense nonlinear layer ℓ combines the two states $\mathbf{h}_T^f$ and $\mathbf{h}_T^b$ and produces an output vector $\mathbf{z} \in \mathbb{R}^{D_z}$. The latter, is the fixed-size, vectorial representation of the MTS.

The decoder operates according to the following map,

$$\tilde{\mathbf{x}}_t = \psi(\mathbf{h}_t, \tilde{\mathbf{x}}_{t-1}, \boldsymbol{\theta}_D), \tag{4}$$

where $\psi(\cdot, \cdot)$ is a stacked RNN with M layers parametrized by $\boldsymbol{\theta}_D$ that operates in a generative mode, processing the previously generated output as a new input. To initialize the decoder, we let its initial state $\mathbf{h}_0 = \mathbf{z}$ and first input $\tilde{\mathbf{x}}_0 = \mathbf{0}$, which corresponds to an "average input" if the MTS are standardized. The decoder iteratively produces outputs for T steps, T being the length of the input MTS. Unequal lengths are naturally handled since the whole architecture is independent of T.

The TKAE is trained end-to-end by means of stochastic gradient descent with scheduled sampling [44]. More specifically, during training the decoder input at time t is, with probability p_s, the decoder output at time $t-1$ (inference mode) and with probability $1-p_s$ the desired output at time $t-1$ (teacher forcing). Since the desired output is not available during the test phase, the decoder generates test data operating only in generative mode ($p_s = 1$). In most of our experiments, scheduled sampling improved the training convergence speed, providing a practical motivation for our choice.

Analogously to standard AEs, RNNs cannot directly process data with missing values, which are thus filled beforehand with some imputed value (0, mean value, last observed value) [45]. However, imputation injects biases in the data that may negatively affect the quality of the representations and conceal potentially useful information contained in the missingness patterns. To overcome these shortcomings, we introduce a *kernel alignment* procedure [20] that allows us to preserve the pairwise similarities of the inputs in the learned representations. These pairwise similarities are encoded in a positive semi-definite matrix $\mathbf{K}$ that is defined by the designer and passed as input to the model. In our case, by choosing the TCK matrix as $\mathbf{K}$, the learned representations will also account for missing data.

Kernel alignment is implemented by an additional regularization term in the loss function (2), which becomes

$$L = L_r + \lambda L_2 + \alpha L_k. \tag{5}$$

L_k is the kernel alignment cost, which takes the form of a normalized Frobenius norm of the difference between two matrices:

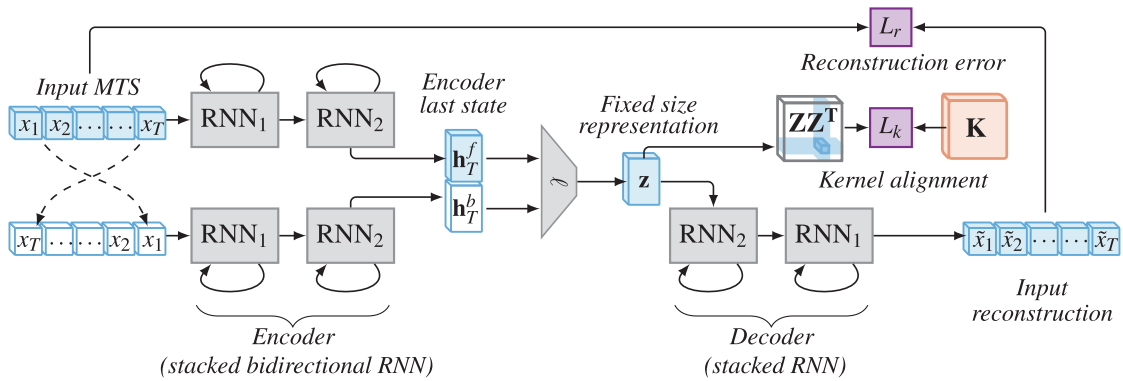


Fig. 1. Schematic representation of TKAE. Inputs are processed by a stacked bidirectional RNN. The last states obtained in forward $\mathbf{h}_T^f$ and backward $\mathbf{h}_T^b$ directions are combined by a dense layer ℓ to produce a fixed-size representation $\mathbf{z}$ of the input. $\mathbf{z}$ is used to initialize the state in the decoder, which is a stacked RNN operating in generative mode and is trained to reproduce inputs by minimizing the reconstruction error $\mathbf{L_r}$. TKAE allows learning similarity-preserving representations of the inputs. In particular, the matrix $\mathbf{ZZ}^T$ containing the dot products of the representations of the MTS in the dataset is aligned, by means of a cost term L_k, to the kernel matrix $\mathbf{K}$. The kernel matrix $\mathbf{K}$ is provided by the user as prior information to condition the representations. In our case, the kernel alignment generates representations whose relationships account for missing data in the input.

$\mathbf{K}$ and $\mathbf{ZZ}^T$, the dot product matrix between the hidden representations $\mathbf{z}$ of the input MTS. More specifically, the L_k term is defined as

$$L_k = \left\| \frac{\mathbf{ZZ}^T}{\|\mathbf{ZZ}^T\|_F} - \frac{\mathbf{K}}{\|\mathbf{K}\|_F} \right\|_F, \tag{6}$$

where $\mathbf{Z} \in \mathbb{R}^{N \times D_z}$ is a matrix of hidden representations relative to the N MTS in the dataset (or, more specifically, in the current mini-batch). Finally, α, $\lambda \geq 0$ are hyperparameters controlling the contribution of alignment and regularization costs in the overall loss function.

3. Experiments

The experimental section is organized as follows.

1. **Quantitative evaluations of the representations in the presence of missing data.** In Section 3.1, we evaluate the effectiveness of the kernel alignment for generating compressed representations in the presence of missing data by computing how accurately the representations are classified. Results show that the kernel alignment with the TCK greatly improves the classification accuracy of the MTS representations in the presence of large amounts of missing data.
2. **Design and evaluation of decoder-based frameworks.** In Section 3.2, we propose two novel frameworks based on the TKAE decoder for (i) imputing missing data and (ii) for one-class classification. Both frameworks exploit not only the TKAE hidden representation but also its decoder, as the results are computed by mapping the compressed representations back to the input space. The proposed frameworks are tested on two different case-studies and compared with other methods.

Experimental setup. In the following, we compare the TKAE with methods for dimensionality reduction: PCA, a standard AE, and other RNN-based architectures. The learned compressed representations have the same dimensionality for all models that are taken into account. Let D_x be the input dimensionality; in TKAE $D_x = V$, as it processes recursively each single time step. On the other hand, the MTS must be unfolded into vectors when processed by PCA and AE. Therefore, in AE and PCA the input dimensionality is $D_x = V \cdot T$. We let D_z be the size of the compressed representations, which corresponds to the number of RNN cells in each TKAE layer, the size of the innermost layer in AE, and the number of principal components in PCA, respectively. In all experiments we use an AE with 3 hidden layers, $\{D_x, 30, D_z, 30, D_x\}$; the number of neurons in the intermediate layers (30) has been set after preliminary experiments and is not a critical hyperparameter (comparable results were obtained using 20 or 40 neurons). As a measure of performance, we consider the MSE between the original test data and their reconstruction as produced by each model. In each experiment, we train the models for 5000 epochs with mini-batches containing 32 MTS using the Adam optimizer [46] with an initial learning rate of 0.001. We independently standardize each variate of the MTS in all datasets. In each experiment, and for each method, we identify the optimal hyperparameters with k-fold cross-validation evaluated on the reconstruction error (or, in general, on the unsupervised loss function) and we report the average results on the test set, obtained in 10 independent runs. We consider only TKAE models with a maximum of three hidden layers of either LSTM or GRU cells, as deeper models generally improve performance slightly at the cost of greater complexity [47]. When kernel alignment is not used ($\alpha = 0$), we refer to the TKAE simply as the TAE.

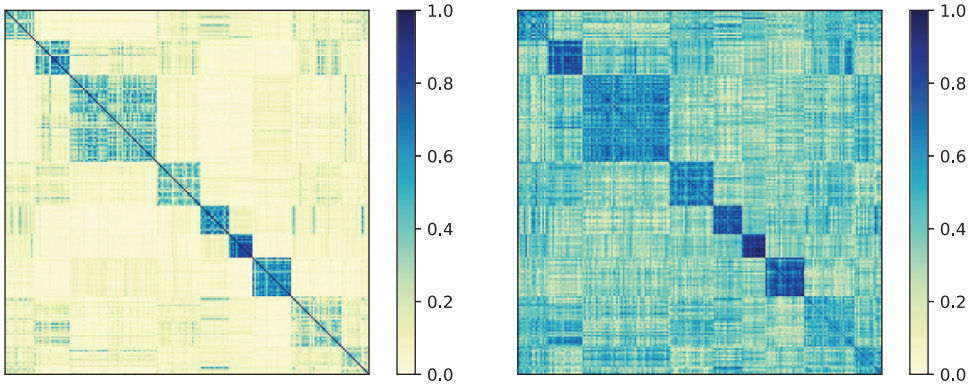
Table 1
Benchmark time series datasets. Column 2 to 5 report the number of attributes, samples in training and test set, and classes, respectively. $T_{\min}$ is the length of the shortest MTS in the dataset and $T_{\max}$ the longest MTS.

Dataset	V	Train	Test	Classes	$T_{\min}$	$T_{\max}$	Source
ECG	1	500	4500	5	140	140	UCR
Libras	2	180	180	15	45	45	[48]
Wafer	6	298	896	2	104	198	UCR
Jp. Vow.	12	270	370	9	7	29	UCI
Arab. Dig.	13	6600	2200	10	4	93	UCI
Auslan	22	1140	1425	95	45	136	UCI
EHR	10	892	223	2	20	20	[49]
Physionet	2	8524	298	4	5	176	[50]

Datasets. We consider several real-world dataset from the UCI and UCR[1] repositories and two medical datasets; details are reported in Table 1. The datasets have been selected in order to cover a wide variety of cases, in terms of training/test sets size, number of variates V, number of classes and (variable) lengths T of the MTS.

The first medical dataset is the *EHR* dataset, which contains blood samples collected over time, extracted from the Electronic Health Records of patients undergoing a gastrointestinal surgery at the University Hospital of North Norway in 2004–2012 [49].

[1] archive.ics.uci.edu/ml/datasets.html, www.cs.ucr.edu/~eamonn/time_series_data.



(a) TCK ($\mathbf{K}$) (b) TKAE ($\mathbf{ZZ}^T$)

Fig. 2. Test set of Jp. Vow. with 80% of missing data. (a) prior $\mathbf{K}$ computed with TCK in input space; (b) dot products $\mathbf{ZZ}^T$ of the representations in TKAE.

Each patient is represented by a MTS of $V = 10$ blood sample measurements collected for $T = 20$ days after surgery. We consider the problem of classifying patients with and without surgical site infections from their blood samples. The dataset consists of $N = 883$ MTS, of which 232 pertain to infected patients. The original MTS contain missing data, corresponding to measurements not collected for a given patient. Data are in random order and the first 80% are used as training set and the rest as test set.

The second medical dataset is *Physionet*, which contains time series of peak-to-peak and RR intervals extracted from ECGs in the 2017 Atrial Fibrillation challenge [50]. The MTS are divided into 4 classes: normal (N), atrial fibrillation (A), other symptoms (O) and noisy records ($\sim$).

3.1. Quantitative evaluations of MTS representations in the presence of missing data

Controlled experiments and sensitivity analysis. To evaluate the effect of kernel alignment when the MTS contain missing data, we perform a controlled experiment where we compare the representations learned by the TAE ($\alpha = 0$) and the TKAE ($\alpha \neq 0$) on the *Jp. Vow.* dataset. This dataset does not originally contain missing data. However, similarly to previous studies [21,48], we inject missing data in a controlled way by randomly removing a certain percentage of values. We vary such percentage from 10% to 90%, evaluating each time the reconstruction MSE and classification accuracy of the TAE and the TKAE encodings using kNN with $k = 3$. We apply zero imputation to replace missing data in the MTS. The TAE and the TKAE are configured with 2 LSTM cells, $p_s = 0.9$ and $\lambda = 0.001$. In the TKAE, $\alpha = 0.1$. In Fig. 2, we show the kernel matrix $\mathbf{K}$ yielded by the TCK and the dot products in $\mathbf{ZZ}^T$ of the representations of the test set when 80% of the data are missing. $\mathbf{ZZ}^T$ is very similar to the TCK matrix, as they are both characterized by a block structure indicating that intra-class similarities in the 9 classes are much higher than inter-class similarities.

Fig. 3 (a) shows how the classification accuracy and reconstruction error of the TAE and the TKAE vary as we increase the amount of missing data.

The classification accuracy (blue lines) does not decrease in the TKAE when the data contain up to 50% missing values and is always higher than in the TAE. When 90% of the data are missing, the TKAE still achieves a classification accuracy of 0.7, while for the TAE it drops to 0.1. We note that the reconstruction MSE decreases for a higher amount of missingness. The reason is that by using imputation, a higher amount of missingness introduces more constant values and, therefore, there is less information to compress.

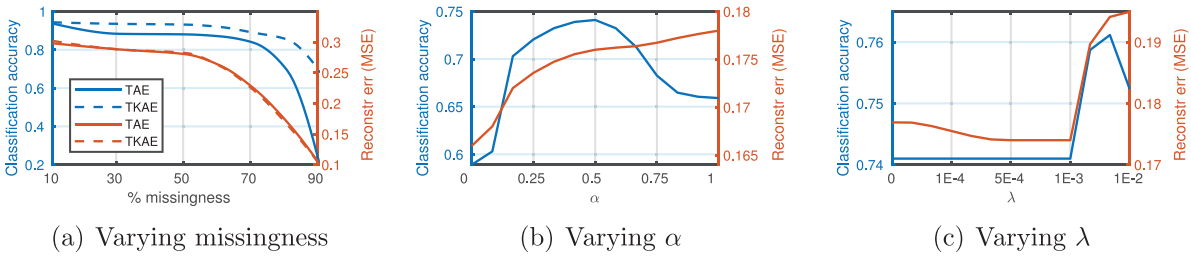
Fig. 3 (b) reports the classification accuracy and reconstruction error, by varying the kernel alignment parameter α when $\lambda = 0$ is fixed and the percentage of missing values is 80%. It is possible to note that there is quite a large interval ranging from 0.25 to 0.7 where the classification accuracy is above 70%. We also observe that the alignment term does not compromise input reconstruction, since the MSE does not significantly change as α is increased.

Finally, Fig. 3(c) reports the classification accuracy and reconstruction error, by varying the L_2 regularization parameter λ, when $\alpha = 0.5$ is fixed and the percentage of missing values is 80%. The small changes in the results demonstrate that the TKAE is not very sensitive to this hyperparameter.

Classification of MTS representations in the presence of missing data. In this second experiment, we analyze the MTS from the EHR, Japanese Vowels, and Arabic Digits datasets. Since the last two datasets are complete, missingness is introduced by removing 80% of the samples. The performance is assessed by classifying the representations of the test set generated by each model. We include in the comparison the representations generated by PCA, a standard AE, the TAE, and the Encoder-Decoder scheme for Anomaly Detection (EncDec-AD), which is a state-of-the-art architecture that generates vectorial representations of MTS [51]. The main differences between EncDec-AD and the TAE, are the lack of the bidirectional encoder in EncDec-AD and a deep architecture in TAE obtained by stacking multiple RNN layers in both the encoder and the decoder. The networks considered for this experiment are configured with the parameters reported in Table 2.

Beside classification accuracy, in Table 3 we also report the F1 score that handles class imbalance. For example, in the EHR data the *infected* class is under-represented compared to *not infected*.

As expected, the AE achieves consistently better results than PCA. In all the experiments, we observe that the TAE performs better than the EncDec-AD, which indicates the importance of the bidirectional encoder to learn MTS representations. Except for the Arabic Digits dataset, the TAE performs also slightly better than the AE. However, when using kernel alignment the performance is boosted, as indicated by the results of the TKAE that are always the best in each task. In particular, we observe that the TKAE representations achieve the best accuracy and a much higher F1 score in the EHR dataset. Fig. 4 depicts the first two principal components of the representations from the EHR dataset learned by the TAE and by the TKAE. It is possible to recognize the effect of the kernel alignment, as the densities of the components relative to different classes become more separated.



(a) Varying missingness (b) Varying α (c) Varying λ

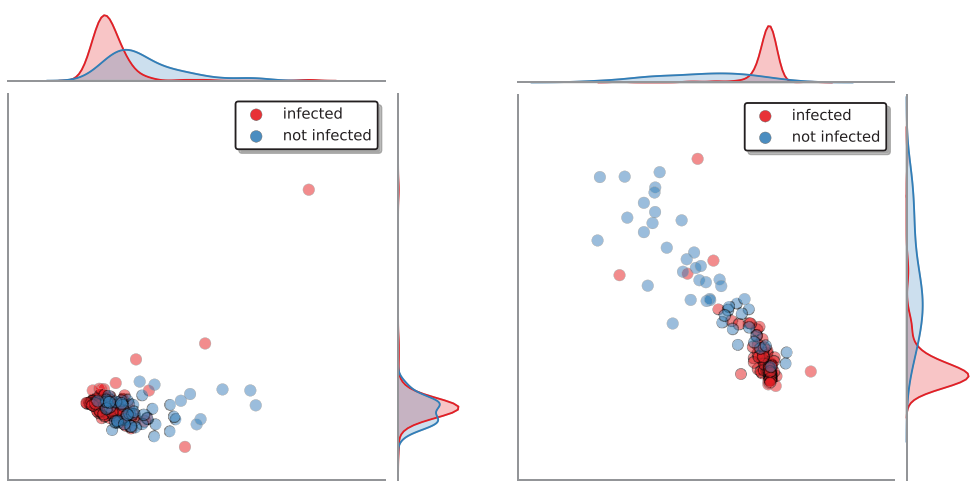
Fig. 3. Classification accuracy (in blue) and reconstruction MSE (in red) on Japanese Vowels dataset. In (a), TAE and TKAE results are reported as a function of the missing values percentage. Panel (b) reports the sensitivity analysis for the parameter α, when $\lambda = 0$ is fixed and the percentage of missing values is 80%. Panel (c) reports the sensitivity analysis for the parameter λ, when $\alpha = 0.5$ is fixed and the percentage of missing values is 80%. (For interpretation of the references to colour in this figure legend, the reader is referred to the web version of this article.)

Table 2
Optimal hyperparameters found with cross-validation. For AE: type of activation function (ψ), l_2 regularization (λ), and tied weights in the decoder (tw). For TAE and TKAE: type of cell $\times$ number of layers, probability of scheduled sampling (p_s), kernel alignment (α), and l_2 regularization (λ).

Dataset	D_z	AE			EncDec-AD, TAE, TKAE	TAE, TKAE		TKAE
		ψ	λ	tw	λ	cell	p_s	α
EHR	10	lin.	0.001	yes	0	GRU $\times$ 2	0.9	0.1
Jp. Vow.	10	lin.	0.001	no	0.001	LSTM $\times$ 2	0.8	0.1
Arab. Dig.	10	lin.	0	yes	0	LSTM $\times$ 2	1.0	0.95

Table 3
Classification of the blood data. F1 score is calculated considering *infected* as "positive" class.

	EHR		Jp. Vow.		Arab. Digits	
Method	Accuracy	F1 score	Accuracy	F1 score	Accuracy	F1 score
PCA	83.5	65.1	76.8	76.7	84.8	84.8
AE	84.6 ± 0.13	67.5 ± 0.34	78.1 ± 0.02	78.2 ± 0.03	85.1 ± 0.01	85.1 ± 0.01
EncDec-AD	82.9 ± 0.07	60.5 ± 0.18	75.9 ± 0.05	76.4 ± 0.05	66.3 ± 0.14	65.1 ± 0.11
TAE	85.3 ± 0.021	68.2 ± 0.22	78.6 ± 0.17	79.1 ± 0.15	73.1 ± 0.09	72.8 ± 0.11
TKAE	**89.9** ± 0.22	**80.2** ± 0.47	**82.4** ± 0.01	**82.6** ± 0.01	**86.8** ± 0.06	**86.7** ± 0.02



(a) TAE (b) TKAE

Fig. 4. PCA and density of the two principal components of the representations yielded by TAE and TKAE on EHR dataset. The densities are computed with a kernel density estimator.

3.2. Decoder-based frameworks

Imputation of missing data. To perform missing data imputation, we modify the loss function of the TKAE slightly and exploit the decoder to impute missing values in the MTS. In the presence of missing data, the reconstruction MSE of the loss function can be modified to account only for non-imputed values,

$$L_r = -\sum_t ((\mathbf{x}_t - \tilde{\mathbf{x}}_t)m_t)^2 / \sum_t m_t, \tag{7}$$

where $m_t = 0$ if $\mathbf{x}_t$ is imputed and 1 otherwise. In this way, the decoder is not constrained to reproduce the values that are imputed and, instead, freely assigns values to the entries that are originally missing. Thus, we can exploit the generalization capability of the decoder to provide an alternative form of imputation, which depends on the nonlinear relationships existing in the training data. A similar principle is followed by denoising AEs (DAEs) [33], as they try to reconstruct the original input from a corrupted version of it, where some entries are randomly removed. Therefore, after

Table 4
Optimal hyperparameters found with cross-validation. For DAE: type of activation function (ψ), λ of l_2 regularization, and tied weights in the decoder (tw). For TKAE: type of cell, number of layers, probability of scheduled sampling (p_s), and λ of l_2 regularization.

Dataset	D_z	DAE			TKAE		
		ψ	λ	tw	cell	p_s	λ
ECG	10	lin.	0	no	GRU × 1	1.0	0
Lbras	5	sig.	0.001	no	LSTM × 2	0.9	0.001
Wafer	10	lin.	0	no	LSTM × 2	1.0	0
Jp. Vow.	10	lin.	0.001	no	LSTM × 2	0.8	0.001
Auslan	10	lin.	0	yes	LSTM × 2	1.0	0

training, DAEs can be exploited to impute missing values on new unseen data [52,53].

We randomly remove approximately 50% of the values from 5 of the datasets described in Table 1 and compare the capability of the TKAE to reconstruct missing values with respect to other imputation techniques. As baselines, we consider mean imputation, last occurrence carried forward (LOCF), and DAE imputation [54]. For the TKAE and the DAE, we use the optimal configurations identified with cross-validation, reported in Table 4. In the TKAE we replace the L_r term with (7) and we set $\alpha = 0.1$. In the DAE, we apply a stochastic corruption of the inputs, by randomly setting input values to 0 with probability 0.5.

In Table 5 we report the MSE and the Pearson correlation (CORR) of the MTS with imputed missing values, with respect to the original MTS. We observe that in 4 of the 5 datasets the TKAE yields the most accurate reconstruction of the true input, followed by the DAE. However, in *Jp. Vow.* LOCF imputation allows retrieval of missing values with the highest accuracy. This can be explained by the fact that in the MTS of the *Jp. Vow.* dataset very similar values are repeated for several time intervals. However, we notice that also in this case the TKAE achieves the second-best result and it outperforms the DAE.

One-class classification. One-class classification and anomaly detection are applied in several domains, including healthcare [55], where non-nominal samples are scarce and often unavailable during training [56]. When dealing with MTS data, RNN-based approaches have been adopted to perform anomaly detection tasks [51,57]. The methods based on dimensionality reduction procedures, such as AEs and energy based models [58,59] rely on the assumption that anomalous samples do not belong to the subspace containing nominal data, which is learned during training. Therefore, the representations generated by the trained model for samples of a new, unseen class will arguably fail to capture important characteristics. Consequently, for those samples an AE would yield large reconstruction errors, which we consider as the classification scores for the new class.

For this task, we consider the real-world data from the *Physionet* dataset. By following a commonly adopted procedure [60], we simulate missing data by randomly removing approximately 50% of the entries in each MTS and then we exclude samples of class A from the training set (which are then considered as non-nominal). We evaluated the performance of the TKAE, the AE, and PCA in detecting class A in a test set containing samples of all classes (N,A,O,∼). As performance measure, we considered the area under ROC curve (AUC) and compared the performance also with two baseline classifiers: one-class SVM (OCSVM) [61] and Isolation Forests (IF) [62]. The optimal configurations are: $D_z = 10$; EncDec-AD, TAE and TKAE with $\lambda = 0$; TAE and TKAE with 1 layer of GRU cells, $p_s = 0.9$; TKAE with $\alpha = 0.2$; AE with non-linear decoder, no tied weights, and $\lambda = 0$; OCSVM with rbf kernel width $\gamma = 0.7$ and $\nu = 0.5$; IF with contamination 0.5. Results in Table 6 show that the TKAE scores the highest AUC.

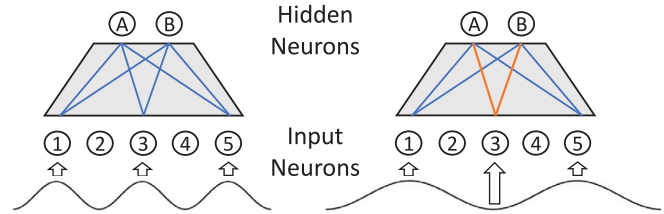


Fig. 5. Periodic inputs with different frequencies generate different activation patterns in AEs. It is not possible to learn connections weights that preserve neurons couplings for each frequency.

4. Comparative analysis of recurrent and feed-forward architectures for learning compressed representations of MTS

So far, we demonstrated the capability of the TKAE to learn *good* representations even in the presence of missing data, thanks to the kernel alignment with the TCK. Another characterizing component of the TKAE is the use of RNNs in both encoder and decoder. Recurrent layers have been successfully applied in *seq2seq* models [26] to encode different types of sequential data, such as text and videos. However, their application to real-valued MTS has been limited so far and it is not clear yet in which cases recurrent AEs work well. Therefore, in this section we investigate when RNNs can represent MTS better than a feed-forward architecture, which processes the whole MTS at once using padding to deal with inputs of variable lengths. We show that in most cases an AE with RNNs encodes well the MTS, which justifies our choice in the design of the TKAE. However, we also report examples of negative results where RNNs fail to process MTS exhibiting certain properties.

Since we want to focus only on the effects of using recurrent layers in generating compressed representations, in the following we do not use kernel alignment, but we consider only the TAE (i.e., TKAE with $\alpha = 0$). Synthetic data are generated to study specific MTS properties in controlled environments.

Time series with different frequencies. Here, we evaluate the capability of the TAE to compress periodic signals having different frequencies and phases. We generate a dataset of sinusoids $y(t) = \sin(a \cdot t + b)$, where a, b are drawn from $\mathcal{N}(0, 1)$ and $t \in [0, 100]$. The proposed task is closely related to the multiple superimposed oscillators, studied in pattern generation and frequency modulation [63]. The training and test sets contain 200 and 1000 samples, respectively. We let $D_z = 5$ and the optimal configurations are: AE with nonlinear decoder and $\lambda = 0.001$; TAE with 2 layers of LSTM cells, $\lambda = 0$, and $p_s = 1.0$. The reconstruction MSE on the test set is 0.41 for PCA, 0.212 for the AE, and 0.013 for the TAE.

Both PCA and the AE process the entire time series at once. This may appear an advantage with respect to the TAE, which stores information in memory for the whole sequence length before yielding the final representation. Nonetheless, the TAE produces a better reconstruction, while the AE (and PCA) is unsuitable for this task. Indeed, in AEs a given time step t in each MTS is always processed by the same input neuron. For periodic signals, the training procedure tries to couple neurons associated to time steps with the same phase, by assigning similar weights to their connections. However, these couplings always change if inputs have different frequencies (see Fig. 5).

Therefore, training in the AE never converges as it is impossible to learn a model that generalizes well for each frequency. On the other hand, thanks to its recurrent architecture, the TAE can

Table 5
MSE and Pearson correlation (CORR) of the MTS where missing values are imputed using different methods, with respect to the original MTS (without missing values). Best and second best results are highlighted in dark and light blue, respectively.

Dataset	Mean Imp.		LOCF		DAE		TKAE	
	MSE	CORR	MSE	CORR	MSE	CORR	MSE	CORR
ECG	0.883	0.702	0.393	0.884	0.157±0.004	0.956±0.001	**0.151**±0.003	**0.956**±0.001
Libras	0.505	0.666	0.085	0.949	0.050±0.001	0.970±0.001	**0.029**±0.002	**0.978**±0.002
Wafer	0.561	0.695	0.226	0.911	0.199±0.017	0.935±0.004	**0.093**±0.007	**0.964**±0.003
Jp. Vow.	0.502	0.699	**0.084**	**0.954**	0.132±0.001	0.926±0.000	0.114±0.003	0.938±0.001
Auslan	0.532	0.613	0.379	0.746	0.145±0.002	0.873±0.005	**0.087**±0.001	**0.941**±0.002

Table 6
AUC obtained by different one-class classification methods in detecting the MTS of atrial fibrillation class, which is not present in the training set.

Method	AUC
OCSVM	0.713
IF	0.662 ± 0.01
PCA	0.707
AE	0.712 ± 0.001
EncDec-AD	0.719 ± 0.007
TAE	0.728 ± 0.005
TKAE	**0.732** ± 0.006

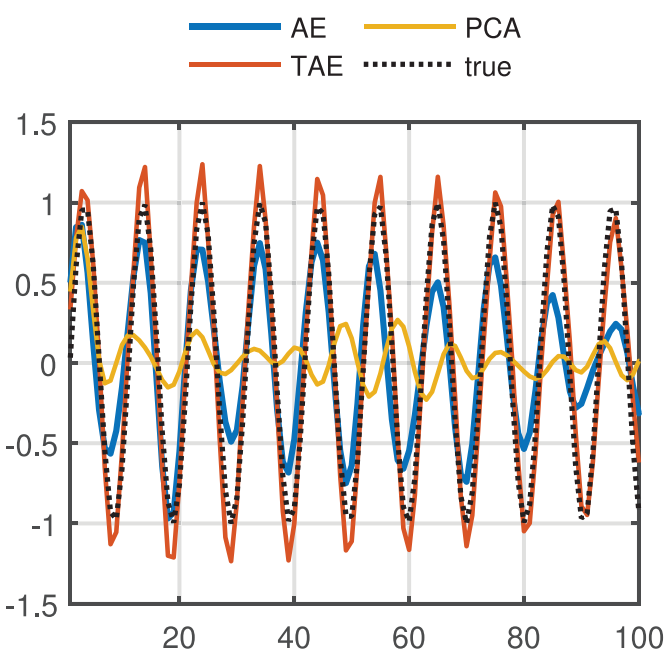


Fig. 6. Reconstructions obtained by PCA, AE, and TAE on a sample sinusoid, whose frequency and phase are randomly chosen.

naturally handle inputs of different frequencies as there is no pairing between structural parameters and time steps.

Fig. 6 shows the reconstruction of one sample time series.

The lower quality of the reconstruction yielded by the AE and by PCA can be immediately noticed. Additionally, since they are unable to reproduce the dynamics of each sample, they rather adopt a more conservative behavior and output signals with lower amplitudes that are closer (in a mean square sense) to the "average" of all the random sinusoids in the dataset.

Time series with variable lengths. While the TAE can process MTS of different length, the standard AE and PCA require inputs of fixed size. The common workaround also followed in this work, is to pad the shorter MTS with zeros [64]. To systematically study the performance of the different methods when the MTS have fixed or variable length, we generate data by integrating the following system of first-order Ordinary Differential Equations (ODE):

Table 7
Average reconstruction MSE of MTS with fixed (`ODEfix`) and variable (`ODEvar`) length.

Dataset	PCA	AE	TAE
`ODEfix`	0.018	**0.004**	0.060
`ODEvar`	0.718	0.676	**0.185**

$$\frac{d\mathbf{y}}{dt} = \mathbf{A}\tanh(\mathbf{y}(t)), \tag{8}$$

where $\mathbf{y} \in \mathbb{R}^V$, $\mathbf{A} \in \mathbb{R}^{V \times V}$ is a matrix with 50% sparsity and elements uniformly drawn in $[-0.5, 0.5]$. To guarantee system stability [65], we set the spectral radius of $\mathbf{A}$ to 0.8. $\tanh(\cdot)$ is applied component-wise and introduces nonlinear dependencies among the variables. A MTS $\mathbf{x} \in \mathbb{R}^{T \times V}$ is obtained by integrating (8) for T steps, starting from a random initial condition $\mathbf{y}(0)$. Since a generic deterministic dynamical system can be described by an ODE system, these synthetic MTS can represent many real data.

We generate two different datasets of MTS with $V = 10$ variables, each one with 400 and 1000 samples for training and test set, respectively. The first, `ODEfix`, contains MTS with same length $T = 90$, while in the second, `ODEvar`, each MTS has a random length $T \in [30, 90]$. We let $D_z = 10$ and compare the reconstruction MSE of PCA, the AE, and the TAE. The optimal configurations for this task are: AE with $\lambda = 0.001$ and linear decoder; TAE with 1 LSTM layer, $\lambda = 0.001$, and $p_s = 0.9$. The average results obtained for 10 independent random generations of the data ($\mathbf{A}$) and initialization of AE and TAE are reported in Table 7.

In `ODEfix`, both the AE and PCA yield almost perfect reconstructions, which is expected due to the simplicity of the task. However, they perform worse in `ODEvar` despite the presence of many padded values and a consequent lower amount (on average) of information to encode in the compressed representation. On the other hand, the TAE naturally deals with variable-length inputs, since once the input sequence terminates its state and model weights during the training are no longer updated.

Dealing with a large number of variates and time steps. To test the ability to learn compressed representations when the number of variates in the MTS increases, starting from (8) we generate four datasets `ODE5`, `ODE10`, `ODE15`, and `ODE20`, obtained by setting $V = \{5, 10, 15, 20\}$. The number of time steps is fixed to $T = 50$ in each dataset. We let $D_z = 10$; TAE is configured with 2 layers of LSTM and $p_s = 0.9$; λ is 0.001 in both the AE and the TAE. We also include in the comparison an AE with tied weights in the (nonlinear) decoder, which has fewer parameters. Reconstruction

Table 8
Average reconstruction MSE on the ODE task for different values of V, obtained by TAE, AE, AE with tied weights (tw) and PCA. For AE and TAE we report the number of trainable parameters (#par). Best results are in bold.

Dataset	TAE		AE		AE (tw)		PCA
	MSE	#par	MSE	#par	MSE	#par	MSE
ODE5	0.019	6130	**0.04**	31,170	0.014	15,870	0.007
ODE10	0.060	6780	**0.04**	61,670	0.071	31,370	0.018
ODE15	**0.072**	7430	0.106	92,170	0.153	46,870	0.174
ODE20	**0.089**	8080	0.121	122,670	0.181	62,370	0.211

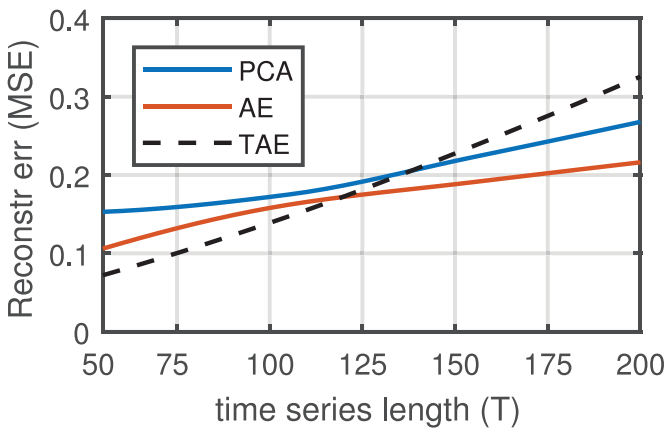


Fig. 7. Reconstruction MSE when increasing length T of MTS in ODE15. TAE performance decreases for large T.

errors are reported in Table 8. We notice that the AE performs well on MTS characterized by low dimensionality, but performance degrades when V assumes larger values. Since the AE processes MTS unrolled into a unidimensional vector, the input size grows quickly as V increases (one additional variable increases the input size by T). Accordingly, the number of parameters in the first dense layer scales-up quickly, possibly leading to overfitting. We also notice that the tied weights regularization, despite halving the number of trainable parameters, degrades performance in each case, possibly because it hinders too much the flexibility of the model. On the other hand, the TAE complexity changes slowly, as only one single neuron is added for an additional input dimension. As a consequence, we conclude that the TAE is the best performing model when the MTS have a large number of variates.

To study the performance as the lengths of MTS increase, we generate 8 datasets with the ODE system (8) by varying $T \in \{50, 75, 100, 125, 150, 175, 200\}$, while keeping $V = 15$ fixed. In Fig. 7, we report the reconstruction errors and note that the TAE performance decays as T increases. RNNs excel in capturing recurring patterns in the input sequence and they can model extremely long sequences whenever they are characterized by a strong periodicity. However, in this case there are no temporal patterns in the data that can be exploited by the RNNs to model the inputs. Therefore, the RNN dynamics do not converge to any fixed point and the modeling task becomes much more difficult as the input length increases.

5. Conclusion

We proposed the temporal kernelized autoencoder, an RNN-based model for representing MTS with missing values as fixed-size vectors. Missing values in MTS are commonly found in domains such as healthcare and derive from measurement errors, incorrect data entry or lack of observations. Through a kernel alignment performed with the time series cluster kernel, a similarity measure designed for MTS with missing data, our method learns compressed representations that preserve pairwise relationships defined in the original input space, even when data are heavily corrupted by missing values.

We showed that the representations learned by the TKAE can be exploited both in supervised and unsupervised tasks. Experimental results, contrasted with other dimensionality reduction techniques on several datasets, showed that the TKAE representations are classified accurately also when the percentage of missing data is high. Through sensitivity analysis, we showed that the kernel alignment has very little impact on the reconstruction error, demonstrating that the TKAE can learn *good* representations even when using the alignment procedure.

After training, only the TKAE encoder is used to generate the representation of the MTS, while the decoder, which is learned as part of the optimization, remain unused. To fully exploit the capabilities of the TKAE architecture, we considered two applications that take advantage of the decoder module. Specifically, we designed two frameworks based on dimensionality reduction and inverse mapping to the input space for imputing missing data and for one-class classification. We showed that by thresholding the reconstruction error of the decoder, the TKAE is able to outperform competing approaches on these tasks.

We concluded our work by investigating which types of MTS are better modeled by a neural network auto-encoder with recurrent layers, rather than with feed-forward ones. Our results showed that in most cases an RNN-based AE is the best architecture to generated good MTS representations. This motivated our design choice for the TKAE. Our analysis revealed that an RNN excels in encoding short MTS with many variables, that are characterized by different lengths or by a varying periodicity. However, when MTS are very long and do not contain temporal patterns that can be modeled by RNNs, better performance can be achieved by replacing recurrent layers in the TKAE with standard dense layers.

Acknowledgments

This work was partially funded by the Norwegian Research Council FRIPRO grant no. 239844 on developing the *Next Generation Learning Machines*. The authors would like to thank Arthur Revhaug, Rolv-Ole Lindsetmo and Knut Magne Augestad, all clinicians currently or formerly affiliated with the gastrointestinal surgery department at the University Hospital of North Norway, for preparing the blood samples dataset. LL gratefully acknowledges partial support of the Canada Research Chairs program.

Appendix A. Details of the TCK algorithm

A MTS $\mathbf{X} \in \mathbb{R}^{V \times T}$ is represented by a sequence of V univariate time series (UTS) of length T, $\mathbf{X} = \{\mathbf{x}_v \in \mathbb{R}^T \mid v = 1, \dots, V\}$, being V and T the dimension and length of $\mathbf{X}$, respectively. Given a dataset of N samples, $\mathbf{X}^{(n)}$ denotes the nth MTS and a binary MTS $\mathbf{R}^{(n)} \in \mathbb{R}^{V \times T}$ describes whether the realisation $x_v^{(n)}(t)$ in $\mathbf{X}$ is observed ($r_v^{(n)}(t) = 1$) or is missing ($r_v^{(n)}(t) = 0$).

DiagGMM. The TCK kernel matrix is built by first fitting G diagonal covariance GMM (DiagGMM) to the MTS dataset. Each DiagGMM g is parametrized by a time-dependent mean $\boldsymbol{\mu}_{gv} \in \mathbb{R}^T$ and a time-constant covariance matrix $\Sigma_g = diag\{\sigma_{g1}^2, \dots, \sigma_{gV}^2\}$, being σ_{gv}^2 the variance of UTS v. Moreover, the data is assumed to be *missing at random*, i.e. the missing elements are only dependent on the observed values. Under these assumptions, missing data can be analytically integrated away [66] and the pdf for each incompletely observed MTS $\{\mathbf{X}, \mathbf{R}\}$ is given by

$$p(\mathbf{X} \mid \mathbf{R}, \Theta) = \sum_{g=1}^{G} \theta_g \prod_{v=1}^{V} \prod_{t=1}^{T} \mathcal{N}(x_v(t) \mid \boldsymbol{\mu}_{gv}(t), \sigma_{gv})^{r_v(t)} \tag{A.1}$$

The conditional probabilities follows from Bayes' theorem,

$$\boldsymbol{\pi}_g = \frac{\theta_g \prod_{v=1}^{V} \prod_{t=1}^{T} \mathcal{N}\left(x_v(t) \mid \boldsymbol{\mu}_{gv}(t), \sigma_{gv}\right)^{r_v(t)}}{\sum_{g=1}^{G} \theta_g \prod_{v=1}^{V} \prod_{t=1}^{T} \mathcal{N}\left(x_v(t) \mid \boldsymbol{\mu}_{gv}(t), \sigma_{gv}\right)^{r_v(t)}}. \tag{A.2}$$

The parameters of the DiagGMM are trained by means of a maximum a posteriori expectation maximization algorithm, as described in [21].

Ensemble generation. To ensure diversity in the ensemble, each GMM model has a different number of components from the interval [2, C] and is trained Q times, using random initial conditions and hyperparameters. Specifically, $\mathcal{Q} = \{q = (q_1, q_2) \mid q_1 = 1, \ldots Q,\ q_2 = 2, \ldots, C\}$ denotes the index set of the initial conditions and hyperparameters (q_1), and the number of components (q_2). Moreover, each DiagGMM is trained on a subset of the original dataset, defined by a random set of the MTS samples, a random set $\mathcal{V}$ of $|\mathcal{V}| \leq V$ variables, and a randomly chosen time segment $\mathcal{T}, |\mathcal{T}| \leq T$. The inner products of the posterior distributions from each mixture component are then added up to build the final TCK kernel matrix. Details are provided in Algorithm 1.

Algorithm 1 TCK kernel training.

Require: Training set of MTS $\{\mathbf{X}^{(n)}\}_{n=1}^{N}$, Q initializations, C maximal number of mixture components.

1: Initialize kernel matrix $\mathbf{K} = \mathbf{0}_{N \times N}$.
2: **for** $q \in \mathcal{Q}$ **do**
3: Compute posteriors $\Pi^{(n)}(q) \equiv (\pi_1^{(n)}, \ldots, \pi_{q_2}^{(n)})^T$, by applying maximum a posteriori expectation maximization~[21] to the DiagGMM with q_2 components and by randomly selecting,

i. hyperparameters $\Omega(q)$,
ii. a time segment $\mathcal{T}(q)$ of length $T_{\min} \leq |\mathcal{T}(q)| \leq T_{\max}$,
iii. attributes $\mathcal{V}(q)$, with cardinality $V_{\min} \leq |\mathcal{V}(q)| \leq V_{\max}$,
iv. a subset of MTS, $\eta(q)$, with $N_{\min} \leq |\eta(q)| \leq N$,
v. initialization of the mixture parameters $\Theta(q)$.

4: Update kernel matrix, $\mathbf{K}_{nm} = \mathbf{K}_{nm} + \frac{\Pi^{(n)}(q)^T \Pi^{(m)}(q)}{\|\Pi^{(n)}(q)\| \|\Pi^{(m)}(q)\|}$.
5: **end for**

Ensure: TCK matrix $\mathbf{K}$.

References

[1] C. Chatfield, The Analysis of Time Series: An Introduction, CRC press, 2016.
[2] F.M. Bianchi, L. Livi, C. Alippi, Investigating echo-state networks dynamics by means of recurrence analysis, IEEE Trans. Neural Netw. Learn. Syst. 29 (2) (2018) 427–439, doi:10.1109/TNNLS.2016.2630802.
[3] M. Längkvist, L. Karlsson, A. Loutfi, A review of unsupervised feature learning and deep learning for time-series modeling, Pattern Recognit. Lett. 42 (2014) 11–24, doi:10.1016/j.patrec.2014.01.008.
[4] B.J. Jain, Making the dynamic time warping distance warping-invariant, Pattern Recognit. 94 (2019) 35–52, doi:10.1016/j.patcog.2019.05.016.
[5] I. Oregi, A. Prez, J.D. Ser, J.A. Lozano, On-line elastic similarity measures for time series, Pattern Recognit. 88 (2019) 506–517, doi:10.1016/j.patcog.2018.12.007.
[6] D. Song, N. Xia, W. Cheng, H. Chen, D. Tao, Deep r-th root of rank supervised joint binary embedding for multivariate time series retrieval, in: Proceedings of the 24th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining, ACM, 2018, pp. 2229–2238.
[7] Z. Che, S. Purushotham, D. Kale, W. Li, M.T. Bahadori, R. Khemani, Y. Liu, Time Series Feature Learning with Applications to Health Care, Springer International Publishing, Cham, pp. 389–409. 10.1007/978-3-319-51394-2_20
[8] Y. Ding, A. Ross, A comparison of imputation methods for handling missing scores in biometric fusion, Pattern Recognit. 45 (3) (2012) 919–933, doi:10.1016/j.patcog.2011.08.002.
[9] A. Farhangfar, L. Kurgan, J. Dy, Impact of imputation of missing values on classification error for discrete data, Pattern Recognit. 41 (12) (2008) 3692–3705, doi:10.1016/j.patcog.2008.05.019.
[10] D.F. Heitjan, S. Basu, Distinguishing "missing at random" and "missing completely at random", Am. Stat. (1996).
[11] R.J.A. Little, D.B. Rubin, Statistical analysis with missing data, John Wiley & Sons, 2014.
[12] Z. Che, S. Purushotham, K. Cho, D. Sontag, Y. Liu, Recurrent neural networks for multivariate time series with missing values, Sci. Rep. 8 (1) (2018) 6085.
[13] Y. Li, S. Wang, Q. Tian, X. Ding, Feature representation for statistical-learning-based object detection: a review, Pattern Recognit. 48 (11) (2015) 3542–3559, doi:10.1016/j.patcog.2015.04.018.
[14] Y. Bengio, A. Courville, P. Vincent, Representation learning: a review and new perspectives, IEEE Trans. Pattern Anal. Mach. Intell. 35 (8) (2013) 1798–1828.
[15] R. Jenssen, Kernel entropy component analysis, IEEE Trans. Pattern Anal. Mach. Intell. 32 (5) (2010) 847–860, doi:10.1109/TPAMI.2009.100.
[16] M. Harandi, M. Salzmann, R. Hartley, Dimensionality reduction on SPD manifolds: the emergence of geometry-aware methods, IEEE Trans. Pattern Anal. Mach. Intell. 40 (1) (2018) 48–62, doi:10.1109/TPAMI.2017.2655048.
[17] K.Ø. Mikalsen, C. Soguero-Ruiz, F.M. Bianchi, R. Jenssen, Noisy multi-label semi-supervised dimensionality reduction, Pattern Recognit. 90 (2019) 257–270, doi:10.1016/j.patcog.2019.01.033.
[18] H. Wang, Q. Zhang, J. Wu, S. Pan, Y. Chen, Time series feature learning with labeled and unlabeled data, Pattern Recognit. 89 (2019) 55–66, doi:10.1016/j.patcog.2018.12.026.
[19] K.S. Tuncel, M.G. Baydogan, Autoregressive forests for multivariate time series modeling, Pattern Recognit. 73 (2018) 202–215, doi:10.1016/j.patcog.2017.08.016.
[20] M. Kampffmeyer, S. Løkse, F.M. Bianchi, R. Jenssen, L. Livi, Deep kernelized autoencoders, in: P. Sharma, F.M. Bianchi (Eds.), Image Analysis, Springer International Publishing, Cham, 2017, pp. 419–430.
[21] K.Ø. Mikalsen, F.M. Bianchi, S. Soguero-Ruiz, R. Jenssen, Time series cluster kernel for learning similarities between multivariate time series with missing data, Pattern Recognit. 76 (2018) 569–581, doi:10.1016/j.patcog.2017.11.030.
[22] A. Graves, A.R. Mohamed, G. Hinton, Speech recognition with deep recurrent neural networks, in: 2013 IEEE International Conference on Acoustics, Speech and Signal Processing, 2013, pp. 6645–6649, doi:10.1109/ICASSP.2013.6638947.
[23] Z. Xing, J. Pei, E. Keogh, A brief survey on sequence classification, SIGKDD Explor. Newsl. 12 (1) (2010) 40–48, doi:10.1145/1882471.1882478.
[24] K. Levin, A. Jansen, B.V. Durme, Segmental acoustic indexing for zero resource keyword search, in: IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP), 2015, pp. 5828–5832, doi:10.1109/ICASSP.2015.7179089.
[25] Y. Chung, C. Wu, C. Shen, H. Lee, L. Lee, Audio word2vec: unsupervised learning of audio segment representations using sequence-to-sequence autoencoder, CoRRarXiv:1603.00982(2016).
[26] I. Sutskever, O. Vinyals, Q.V. Le, Sequence to sequence learning with neural networks, in: Z. Ghahramani, M. Welling, C. Cortes, N.D. Lawrence, K.Q. Weinberger (Eds.), Advances in Neural Information Processing Systems 27, Curran Associates, Inc., 2014, pp. 3104–3112.
[27] S.R. Bowman, L. Vilnis, O. Vinyals, A.M. Dai, R. Józefowicz, S. Bengio, Generating sentences from a continuous space, CoRRarXiv:1511.06349(2015).
[28] B. Su, S. Lu, Accurate recognition of words in scenes without character segmentation using recurrent neural network, Pattern Recognit. 63 (2017) 397–405, doi:10.1016/j.patcog.2016.10.016.
[29] N. Srivastava, E. Mansimov, R. Salakhutdinov, Unsupervised learning of video representations using LSTMs, in: Proceedings of the 32nd International Conference on International Conference on Machine Learning - Volume 37, in: ICML'15, JMLR.org, 2015, pp. 843–852.
[30] G.E. Hinton, R.R. Salakhutdinov, Reducing the dimensionality of data with neural networks, Science 313 (5786) (2006) 504–507, doi:10.1126/science.1127647.
[31] Y. Bengio, Learning deep architectures for AI, Found. Trends Mach. Learn. 2 (1) (2009) 1–127, doi:10.1561/2200000006.
[32] D. Erhan, Y. Bengio, A. Courville, P.-A. Manzagol, P. Vincent, S. Bengio, Why does unsupervised pre-training help deep learning? J. Mach. Learn. Res. 11 (2010) 625–660.
[33] P. Vincent, H. Larochelle, I. Lajoie, Y. Bengio, P.-A. Manzagol, Stacked denoising autoencoders: learning useful representations in a deep network with a local denoising criterion, J. Mach. Learn. Res. 11 (Dec) (2010) 3371–3408.
[34] A. Makhzani, J. Shlens, N. Jaitly, I. Goodfellow, B. Frey, Adversarial autoencoders, arXiv:1511.05644(2015).
[35] W.W.Y. Ng, G. Zeng, J. Zhang, D.S. Yeung, W. Pedrycz, Dual autoencoders features for imbalance classification problem, Pattern Recognit. (2016).
[36] J.L. Elman, Finding structure in time, Cogn. Sci. 14 (2) (1990) 179–211.
[37] F.M. Bianchi, E. Maiorino, M.C. Kampffmeyer, A. Rizzi, R. Jenssen, Recurrent Neural Networks for Short-Term Load Forecasting: An Overview and Comparative Analysis, Springer, 2017.
[38] D. Bahdanau, K. Cho, Y. Bengio, Neural machine translation by jointly learning to align and translate, CoRRarXiv:1409.0473(2014).
[39] Y. Qin, D. Song, H. Chen, W. Cheng, G. Jiang, G. Cottrell, A dual-stage attention-based recurrent neural network for time series prediction, arXiv:1704.02971(2017).
[40] K. Cho, B. Van Merriënboer, C. Gulcehre, D. Bahdanau, F. Bougares, H. Schwenk, Y. Bengio, Learning phrase representations using RNN encoder-decoder for statistical machine translation, arXiv:1406.1078(2014).
[41] S. Hochreiter, J. Schmidhuber, Long short-term memory, Neural Comput. 9 (8) (1997) 1735–1780.
[42] J. Chung, C. Gulcehre, K. Cho, Y. Bengio, Empirical evaluation of gated recurrent neural networks on sequence modeling, arXiv:1412.3555(2014).
[43] M. Schuster, K.K. Paliwal, Bidirectional recurrent neural networks, IEEE Trans. Signal Process. 45 (11) (1997) 2673–2681.
[44] S. Bengio, O. Vinyals, N. Jaitly, N. Shazeer, Scheduled sampling for sequence prediction with recurrent neural networks, in: Proceedings of the 28th In-

ternational Conference on Neural Information Processing Systems, MIT Press, Cambridge, MA, USA, 2015, pp. 1171–1179.
[45] J.L. Schafer, J.W. Graham, Missing data: our view of the state of the art., Psychol. Methods 7 (2) (2002) 147.
[46] D. Kingma, J. Ba, Adam: a method for stochastic optimization, arXiv:1412.6980(2014).
[47] N. Reimers, I. Gurevych, Optimal hyperparameters for deep LSTM-networks for sequence labeling tasks, arXiv:1707.06799(2017).
[48] M.G. Baydogan, G. Runger, Time series representation and similarity based on local autopatterns, Data Min. Knowl. Discov. 30 (2) (2016) 476–509, doi:10.1007/s10618-015-0425-y.
[49] K.Ø. Mikalsen, C. Soguero-Ruiz, F.M. Bianchi, A. Revhaug, R. Jenssen, An unsupervised multivariate time series kernel approach for identifying patients with surgical site infection from blood samples, ArXiv e-prints (2018).
[50] G. Clifford, C. Liu, B. Moody, L. Lehman, I. Silva, Q. Li, A.E. Johnson, R.G. Mark, AF Classification from a short single lead ecg recording: the physionet computing in cardiology challenge 2017, Comput Cardiol (2010) (2017).
[51] P. Malhotra, A. Ramakrishnan, G. Anand, L. Vig, P. Agarwal, G. Shroff, Lstm-based encoder-decoder for multi-sensor anomaly detection, arXiv:1607.00148(2016).
[52] B.K. Beaulieu-Jones, C.S. Greene, Semi-supervised learning of the electronic health record for phenotype stratification, J. Biomed. Inform. 64 (2016) 168–178, doi:10.1016/j.jbi.2016.10.007.
[53] L. Gondara, K. Wang, Multiple imputation using deep denoising autoencoders, arXiv:1705.02737(2017).
[54] B.K. Beaulieu-Jones, J.H. Moore, Missing Data Imputation in the Electronic Health Record Using Deeply Learned autoencoders, World Scientific, pp. 207–218. 10.1142/9789813207813_0021
[55] D. Carrera, B. Rossi, P. Fragneto, G. Boracchi, Online anomaly detection for long-term ecg monitoring using wearable devices, Pattern Recognit. 88 (2019) 482–492, doi:10.1016/j.patcog.2018.11.019.
[56] I. Irigoien, B. Sierra, C. Arenas, Towards application of one-class classification methods to medical data, Sci. World J. (2014), doi:10.1155/2014/730712.
[57] C. Zhang, D. Song, Y. Chen, X. Feng, C. Lumezanu, W. Cheng, J. Ni, B. Zong, H. Chen, N.V. Chawla, A deep neural network for unsupervised anomaly detection and diagnosis in multivariate time series data, arXiv:1811.08055(2018).
[58] S. Zhai, Y. Cheng, W. Lu, Z. Zhang, Deep structured energy based models for anomaly detection, in: Proceedings of the 33rd International Conference on International Conference on Machine Learning - Volume 48, in: ICML'16, JMLR.org, 2016, pp. 1100–1109.
[59] D. Chakraborty, V. Narayanan, A. Ghosh, Integration of deep feature extraction and ensemble learning for outlier detection, Pattern Recognit. 89 (2019) 161–171, doi:10.1016/j.patcog.2019.01.002.
[60] J.L. Peugh, C.K. Enders, Missing data in educational research: a review of reporting practices and suggestions for improvement, Rev. Educ. Res. 74 (4) (2004) 525–556, doi:10.3102/00346543074004525.
[61] Y. Guerbai, Y. Chibani, B. Hadjadji, The effective use of the one-class SVM classifier for handwritten signature verification based on writer-independent parameters, Pattern Recognit. 48 (1) (2015) 103–113, doi:10.1016/j.patcog.2014.07.016.
[62] R. Domingues, M. Filippone, P. Michiardi, J. Zouaoui, A comparative evaluation of outlier detection algorithms: experiments and analyses, Pattern Recognit. 74 (2018) 406–421, doi:10.1016/j.patcog.2017.09.037.
[63] D. Sussillo, L.F. Abbott, Generating coherent patterns of activity from chaotic neural networks, Neuron 63 (4) (2009) 544–557, doi:10.1016/j.neuron.2009.07.018.
[64] M.E. Mann, Smoothing of climate time series revisited, Geophys. Res. Lett. 35 (16) (2008), doi:10.1029/2008GL034716.
[65] F.M. Bianchi, L. Livi, C. Alippi, R. Jenssen, Multiplex visibility graphs to investigate recurrent neural network dynamics, Sci. Rep. 7 (2017) 44037.
[66] D.B. Rubin, Inference and missing data, Biometrika 63 (3) (1976) 581–592.

Filippo M. Bianchi received the B.Sc. in computer engineering (2009), M.Sc. in Artificial Intelligence and Robotics (2012) and PhD in Machine Learning (2016) from Sapienza University, Rome, Italy. From 2014 he worked 2 years as research assistant in Computer Science at Ryerson University, Toronto, Canada. In 2017 he worked as scientific collaborator at USI, Lugano, Switzerland. He's currently a postdoc at Machine Learning group at UiT, Tromsø, Norway. His research interests in machine learning and pattern recognition include deep learning with a special focus on recurrent neural networks and autoencoders, reservoir computing, clustering, and graph matching.